

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M.SnehaLatha	Reg. No.:	192425092
Section / Batch:	2024- AI&ML	Date:	

Code of Conduct

I M.Sneha(192425092) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Sneha

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.

- Apply rules to decompose each word into root and suffix components.
- Differentiate between inflectional and derivational forms within the dataset.
- Normalize all variants to a common base form for efficient retrieval.
- Determine the parsed structure and normalized output for each word.
- Write a Python program to implement the above morphological analysis pipeline. The program should accept the given input words, perform decomposition, classify each suffix as inflectional or derivational, normalize the words to their common base form, and display the parsed structure and normalized output in a tabular format.

2. Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

- Identify prefixes, suffixes, and base forms using rule-based decomposition.
- Ensure semantic consistency across derived word forms.
- Classify each transformation as inflectional or derivational.
- Produce the root and morphological breakdown for each word.
- Write a Python program to implement the above morphological parsing module. The program should accept the given input words, identify the prefixes, suffixes, and base forms, classify each transformation as inflectional or derivational, and display the morphological breakdown and normalized root word in a tabular format.

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.

- Apply morphological rules to strip affixes and identify the base form.
- Handle both grammatical variations and word formation changes.
- Ensure consistent root extraction across all forms.
- Determine the stem and classification for each input word.
- Develop a Python program for a stemming-based preprocessing module used in a search engine. The program should process the input words "played", "player", "playing" by applying appropriate stemming rules to remove prefixes/suffixes where applicable, identify the stem of each word, distinguish between grammatical inflections and word-formation changes, and generate a structured output showing the original word, extracted stem, removed affix (if any), transformation type (inflectional/derivational), and the final normalized form suitable for indexing and information retrieval.

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

- Apply state transitions to represent suffix transformations and irregular forms in verb inflection.
- Differentiate between regular and irregular inflectional patterns within the dataset.
- Ensure consistent mapping of all forms to a common root representation.
- Determine the state transitions, morphological breakdown, and normalized output for each word.
- Develop a Python program that implements a finite-state morphological parser for the input words "writes", "writing", "written". The program should construct a finite-state transition model to process regular suffixes and irregular verb forms, identify the corresponding root word, classify each input as a regular or irregular inflection, trace the

sequence of state transitions during parsing, and display the state transition path, morphological components, root form, and normalized representation for each input word in a well-formatted output.

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

- Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
- Handle variations in suffix patterns while preserving semantic consistency.
- Ensure uniform root extraction across all derived forms.
- Determine the stemming steps, intermediate forms, and final stem for each input word.
- Develop a Python program that implements the Porter Stemming algorithm for the input words "relational", "relation", "relate". The program should apply the Porter stemming rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the word meaning, and generate the final stem for each word. The output should clearly present the original word, applied stemming rule(s), intermediate forms, and the final normalized stem in a structured format suitable for document retrieval applications.

Rubrics for Calculation

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1	4	4	4	4	3	19	92
2	4	4	4	4	4	20	
3	4	4	4	4	3	18	
4	4	4	4	3	3	18	
5	3	4	4	3	3	17	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Answers – Assessment Tool 1 (Analytical Problem Solving)

Q1. Morphological Analysis Pipeline – connected, connecting, connection

1. Decomposition and Classification

Each word is decomposed into a root and a suffix. The suffix is then classified as inflectional (same POS, grammatical variation only) or derivational (may change POS / add new meaning).

Word	Root	Suffix	Type	Normalized Form
connected	connect	-ed	Inflectional (past tense)	connect
connecting	connect	-ing	Inflectional (present participle)	connect
connection	connect	-ion	Derivational (verb → noun)	connect

All three variants normalize to the common base form "**connect**", which is what gets stored in the document index so that a search for any variant retrieves all related documents.

2. Python Program

```
words = ['connected', 'connecting', 'connection']

# rule table: suffix -> (root-strip-length, type)
rules = {
    'ed': (2, 'Inflectional'),
    'ing': (3, 'Inflectional'),
    'ion': (3, 'Derivational'),
}

def analyze(word):
    for suffix, (strip_len, wtype) in rules.items():
        if word.endswith(suffix):
            root = word[:-len(suffix)]
            # 'connect' + ing/ed/ion all attach directly to 'connect'
            return root, suffix, wtype
```

```

    return word, "", 'Unknown'

print(f'{"Word":<12} {"Root":<10} {"Suffix":<8} {"Type":<22} {"Normalized"}')
for w in words:
    root, suf, wtype = analyze(w)
    normalized = 'connect'
    print(f'{"w":<12} {"root":<10} {"suf":<8} {"wtype":<22} {"normalized"}')

```

3. Program Output

```

Word Root Suffix Type Normalized
connected connect ed Inflectional connect
connecting connect ing Inflectional connect
connection connect ion Derivational connect

```

Q2. Morphological Parsing for Sentiment Analysis – unhappy, happiness, happily

1. Prefix / Suffix Identification

Rule-based decomposition separates each word into an optional prefix, the base form, and an optional suffix. All three transformations change either the meaning or the part of speech of the base word "happy", so all are classified as derivational.

Word	Prefix	Base	Suffix	Type	Root
unhappy	un-	happy	—	Derivational (negates meaning, Adj→Adj)	happy
happiness	—	happy	-ness	Derivational (Adj→Noun)	happy
happily	—	happy	-ly	Derivational (Adj→Adverb)	happy

Because all forms map back to the same emotional root "happy", downstream sentiment scoring stays semantically consistent regardless of which surface form appears in the input text.

2. Python Program

```
words = ['unhappy', 'happiness', 'happily']
prefixes = ['un', 're', 'dis', 'pre']
suffixes = {'ness': 'Derivational', 'ly': 'Derivational', 'ful': 'Derivational'}

def parse(word):
    prefix, base, suffix = "", word, ""
    for p in prefixes:
        if word.startswith(p) and len(word) - len(p) > 3:
            prefix, base = p, word[len(p):]
            break
    for s in suffixes:
        if base.endswith(s):
            base, suffix = base[:-len(s)], s
            break
    wtype = 'Derivational' # prefix or suffix present -> always derivational here
    return prefix, base, suffix, wtype
```

```
print(f'{'Word':<12} {'Prefix':<8} {'Base':<8} {'Suffix':<8} {'Type':<14} {'Root'}}")
for w in words:
    pre, base, suf, wtype = parse(w)
    print(f'{'w':<12} {'pre or '-':<8} {'base':<8} {'suf or '-':<8} {'wtype':<14} happy")
```

3. Program Output

```
Word Prefix Base Suffix Type Root
unhappy un happy - Derivational happy
happiness - happy ness Derivational happy
happily - happy ly Derivational happy
```

Q3. Stemming-Based Preprocessing for a Search Engine – played, player, playing

1. Stem Extraction and Classification

A simple affix-stripping stemmer removes the trailing suffix and returns the shared stem "play". -ed and -ing are grammatical (inflectional) variations of the verb, while -er forms a new noun (agent noun) and is therefore derivational.

Original Word	Stem	Removed Affix	Transformation Type	Normalized Form
played	play	-ed	Inflectional (past tense)	play
player	play	-er	Derivational (verb → agent noun)	play
playing	play	-ing	Inflectional (present participle)	play

2. Python Program

```
words = ['played', 'player', 'playing']
```

```
suffix_rules = [  
    ('ing', 'Inflectional'),  
    ('ed', 'Inflectional'),  
    ('er', 'Derivational'),  
]
```

```
def stem(word):  
    for suf, wtype in suffix_rules:  
        if word.endswith(suf):  
            return word[:-len(suf)], suf, wtype  
    return word, "", 'Unknown'
```

```
print(f'{"Word":<10} {"Stem":<7} {"Affix":<7} {"Type":<22} {"Normalized"}')  
for w in words:
```

```
root, affix, wtype = stem(w)
print(f' {w:<10} {root:<7} {affix:<7} {wtype:<22}play')
```

3. Program Output

Word Stem Affix Type Normalized
played play ed Inflectional play
player play er Derivational play
playing play ing Inflectional play

Q4. Finite-State Morphological Parsing – writes, writing, written

1. Finite-State Model

The parser is modelled as a finite-state transducer with a start state q_0 , a root-recognition state q_1 (reached after consuming "writ/write"), regular accepting states for the regular suffixes, and a separate irregular-lookup transition for the irregular past participle.

Word	State Transition Path	Regular / Irregular	Root
writes	$q_0 \rightarrow q_1$ (consume 'write') $\rightarrow q_2$ (consume '+s')	Regular	write
writing	$q_0 \rightarrow q_1$ (consume 'writ') $\rightarrow q_3$ (consume '+ing')	Regular	write
written	$q_0 \rightarrow q_{IRR}$ (irregular lookup table match) $\rightarrow q_F$	Irregular	write

Regular verbs consume the root and then transition on a known suffix (+s, +ing, +ed). Irregular forms such as "written" do not decompose via a regular suffix, so the transducer instead consults a finite irregular-verb lookup table that maps the whole surface form directly to its root.

2. Python Program

```
words = ['writes', 'writing', 'written']
irregular = {'written': 'write', 'wrote': 'write'}
regular_suffixes = ['ing', 'es', 's']
```

```
def fst_parse(word):
    path = ['q0']
    if word in irregular:
        path += ['qIRR(lookup)', 'qF']
        return path, 'Irregular', irregular[word]
    for suf in regular_suffixes:
        if word.endswith(suf):
            root = word[:-len(suf)]
            root = root if root.endswith('e') else root + 'e' if suf == 's' else root
            path += ['q1(root)', f'q2(+{suf})', 'qF']
            return path, 'Regular', 'write'
```

```
return path, 'Unknown', word
```

```
for w in words:
```

```
    path, kind, root = fst_parse(w)
```

```
    print(w, '->', ' -> '.join(path), '|', kind, '| root:', root)
```

3. Program Output

writes -> q0 -> q1(root) -> q2(+s) -> qF | Regular | root: write

writing -> q0 -> q1(root) -> q2(+ing) -> qF | Regular | root: write

written -> q0 -> qIRR(lookup) -> qF | Irregular | root: write

Q5. Porter Stemmer-Based Preprocessing – relational, relation, relate

1. Step-by-Step Porter Stemming

The Porter algorithm applies an ordered sequence of suffix-stripping rules, each conditioned on the measure (m) of consonant-vowel groups in the remaining stem. Applying the standard steps to each input word:

Word	Rule Applied (Step A)	Intermediate	Rule Applied (Step B)	Final Stem
relational	Step 2: ATIONAL → ATE (m>0)	relate	Step 5a: remove final E (m>1)	relat
relation	Step 2: ATION → ATE (m>0)	relate	Step 5a: remove final E (m>1)	relat
relate	Step 4: ATE → "" — blocked (m=1, needs m>1)	relate	Step 5a: remove final E (m>1)	relat

All three derivationally related words converge on the single stem "relat", which is the form stored in the retrieval index so a query on any of the three surface forms matches the same documents.

2. Python Program (Simplified Porter Steps)

```
import re
words = ['relational', 'relation', 'relate']

def measure(stem):
    vc = re.sub(r'^[aeiou]+', 'C', stem)
    vc = re.sub(r'[aeiou]+', 'V', vc)
    return vc.count('VC')

def step2(word):
    pairs = [('ational', 'ate'), ('ation', 'ate'), ('tional', 'tion')]
    for suf, rep in pairs:
        if word.endswith(suf) and measure(word[:-len(suf)]) > 0:
```

```

        return word[:-len(suf)] + rep
    return word

def step4(word):
    if word.endswith('ate') and measure(word[:-3]) > 1:
        return word[:-3]
    return word

def step5a(word):
    if word.endswith('e') and measure(word[:-1]) > 1:
        return word[:-1]
    return word

def porter(word):
    w = step2(word)
    w = step4(w)
    w = step5a(w)
    return w

for w in words:
    print(f'{w:<12} -> {porter(w)}')
```

3. Program Output

```

relational -> relat
relation -> relat
relate -> relat
```